

Search Algorithms: Worked Answers

BFS, DFS, Dijkstra, and A*

Original questions: requirements/BFS-DFS-Dijkstra-Astar-homework.pdf, pp. 1–10.

BFS1 (p. 1): Number of states searched

Use the initial board $\begin{bmatrix} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & \square & 5 \end{bmatrix}$. Number the root as depth 0, and exclude repeated states, including the immediate reversal of the previous move. The numbers of distinct states at depths 0 through 5 are:

Depth d	0	1	2	3	4	5
States N_d	1	3	5	10	14	28

The blank initially lies at an edge position, giving three first moves. Each corner blank has two legal moves, each edge blank has three, and the center blank has four. Excluding the move back to the parent gives the following layer calculation:

Depth	Corner blanks	Edge blanks	Center blanks	Total
0	0	1	0	1
1	2	0	1	3
2	0	5	0	5
3	8	0	2	10
4	0	14	0	14
5	20	0	8	28

For example, depth 2 has $2(2 - 1) + (4 - 1) = 5$ states; depth 3 has $5(3 - 1) = 10$ states. Through depth 5, these generated configurations are distinct after parent reversals are excluded.

BFS examines every shallower layer before depth 5. With the question’s assumption that half of depth 5 is searched, the total is

$$N = 1 + 3 + 5 + 10 + 14 + \frac{28}{2} = \boxed{47 \text{ states}}.$$

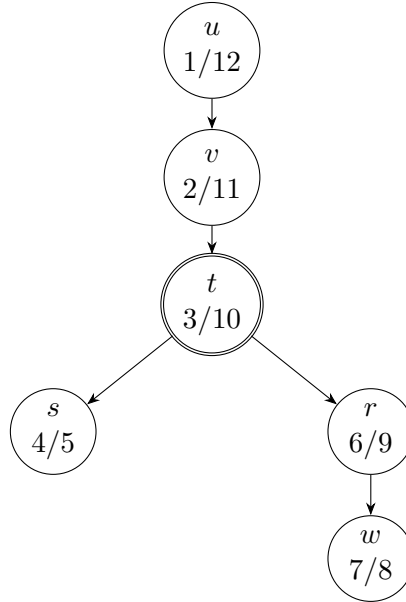
This counts the root and the goal among the states examined. The printed label “27” beside the goal is a diagram label, not a BFS examination count. The answer uses the stated halfway assumption; it is not a claim about a particular implementation’s goal-test ordering or total generated queue entries.

DFS1 (p. 2): Search tree and backtracking

Follow the recursive order illustrated on the question sheet:

$$u \rightarrow v \rightarrow t \rightarrow s, \quad \text{return to } t, \quad t \rightarrow r \rightarrow w.$$

The tree edges are $(u, v), (v, t), (t, s), (t, r), (r, w)$.



Each pair is discovery/finish time. After s finishes, DFS returns to t , which has an unvisited neighbor r . Thus $\boxed{t}$ is the backtracking branch point where a new route starts. The dead ends are s and w .

The complete return sequence is

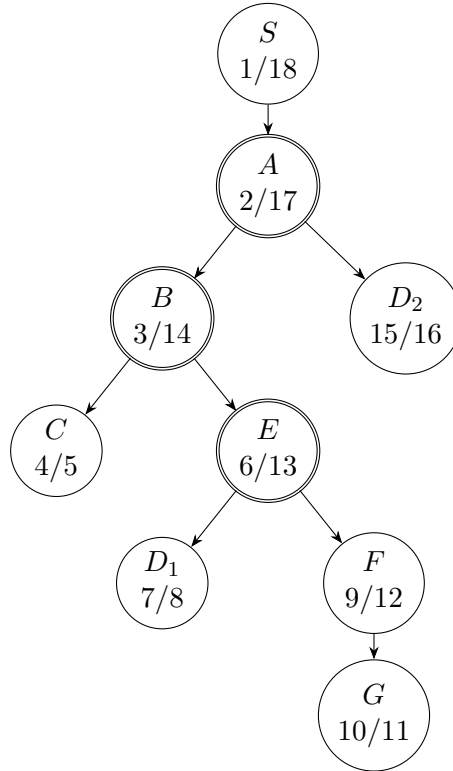
$$s \rightsquigarrow t, \quad w \rightsquigarrow r \rightsquigarrow t \rightsquigarrow v \rightsquigarrow u.$$

If “backtrack nodes” means all nodes receiving a recursive return, they are t, r, v, u ; if it means a return point that starts a different unvisited branch, it is t alone. This distinction is also used in DFS2 and DFS3.

The non-tree edges (s, u) and (w, v) connect descendants to ancestors, so they are back edges in this undirected DFS. An undirected graph has only tree and back edges under the usual DFS edge classification.

DFS2 (p. 3): Discovery and finish times

Reading the supplied tree literally. The left diagram contains two occurrences of D . Label the one below E as D_1 and the one directly below A as D_2 . Treating these as separate tree occurrences and visiting children left to right gives:



The branch points at which DFS returns and begins another branch are

$$C \rightsquigarrow \boxed{B} \rightarrow E, \quad D_1 \rightsquigarrow \boxed{E} \rightarrow F, \quad G \rightsquigarrow F \rightsquigarrow E \rightsquigarrow B \rightsquigarrow \boxed{A} \rightarrow D_2.$$

The dead-end occurrences are C, D_1, G, D_2 . Every internal node eventually receives a recursive return: S, A, B, E, F .

If the intended algorithm is graph DFS on the right-hand graph: D is discovered only once, below E . The later edge $A-D$ does not create another tree node. The corrected timestamps are

$$S : 1/16, \quad A : 2/15, \quad B : 3/14, \quad C : 4/5, \quad E : 6/13, \quad D : 7/8, \quad F : 9/12, \quad G : 10/11.$$

Only B and E start a new unvisited branch after backtracking. The two readings are separated because the supplied tree repeats the graph vertex D .

DFS3 (p. 4): Continue after the dead end

Use (r, c) for row and column. Keep all previously visited nodes marked. The question does not fix the order of the remaining unvisited neighbors, so the following is one valid continuation.

First backtracking point. At $(3, 2)$, all neighbors have already been visited. Backtrack along the active DFS stack:

$$(3, 2) \rightsquigarrow (2, 2) \rightsquigarrow (1, 2) \rightsquigarrow (1, 1) \rightsquigarrow (2, 1) \rightsquigarrow (3, 1) \rightsquigarrow (4, 1).$$

The first node with an unvisited neighbor is $bf_1 = (4, 1)$. Continue into the lower-left pocket:

$$(4, 1) \rightarrow (5, 1) \rightarrow (6, 1) \rightarrow (6, 2).$$

At $(6, 2)$, its neighbors $(6, 1)$, $(5, 2)$, $(6, 3)$ are all visited, so it is another dead end.

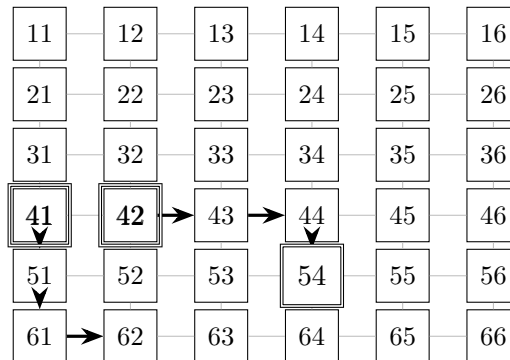
Second backtracking point. Return via

$$(6, 2) \rightsquigarrow (6, 1) \rightsquigarrow (5, 1) \rightsquigarrow (4, 1) \rightsquigarrow (4, 2).$$

Now $bf_2 = (4, 2)$ has the unvisited neighbor $(4, 3)$. Continue:

$$(4, 2) \rightarrow (4, 3) \rightarrow (4, 4) \rightarrow (5, 4).$$

The goal is reached, so stop.



The arrows show only newly explored edges. Double boxes mark $bf_1 = 41$, $bf_2 = 42$, and the goal 54. The final start-to-goal stack is

$$\begin{aligned} & 33 \rightarrow 23 \rightarrow 13 \rightarrow 14 \rightarrow 15 \rightarrow 25 \rightarrow 35 \rightarrow 45 \rightarrow 55 \\ & \rightarrow 65 \rightarrow 64 \rightarrow 63 \rightarrow 53 \rightarrow 52 \rightarrow 42 \rightarrow 43 \rightarrow 44 \rightarrow 54. \end{aligned}$$

Here 33 means $(3, 3)$, and similarly for the other labels. The dead-end side branches are removed from the final path.

DFS4 (p. 5): DFS starting from v

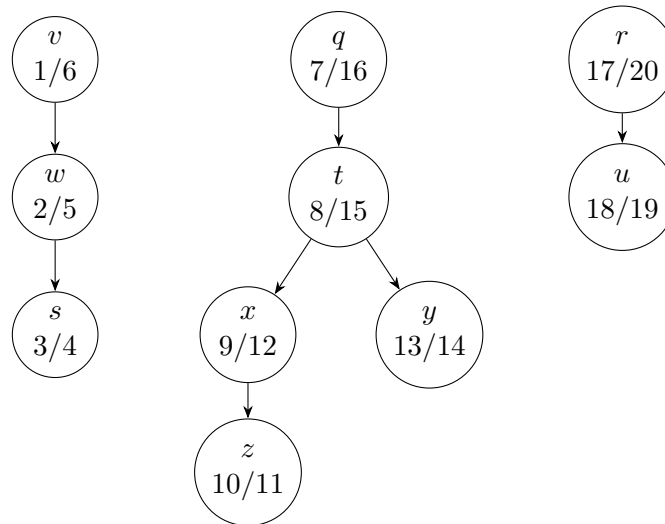
Start with v . After its component finishes, scan the remaining vertices alphabetically. Use alphabetical adjacency lists throughout.

Vertex	Outgoing neighbors	Vertex	Outgoing neighbors
q	s, t, w	r	u, y
s	v	t	x, y
u	y	v	w
w	s	x	z
y	q	z	x

The discovery order is

 $v, w, s, q, t, x, z, y, r, u$

The resulting DFS forest has three roots, v, q, r :



Finish order: $s, w, v, z, x, y, t, q, u, r$. The non-tree edges are:

- Back edges: $s \rightarrow v$, $z \rightarrow x$, $y \rightarrow q$.
- Cross edges: $q \rightarrow s$, $q \rightarrow w$, $u \rightarrow y$, $r \rightarrow y$.

Only v, w, s are reachable from the initial root v ; the outer DFS loop is needed to visit the rest.

DFS5 (p. 6): Topological ordering

One valid topological ordering is

$p, n, o, s, m, r, y, v, x, w, z, u, q, t$.

It can be obtained by running DFS with alphabetical outer and adjacency order, then reversing the finish order. Every directed edge goes from an earlier vertex to a later vertex:

Vertex	Its outgoing neighbors, all later in the ordering
p	o, s, z
n	o, q, u
o	r, s, v
s	r
m	q, r, x
r	u, y
y	v
v	w, x
w	z
u	t
q	t

The vertices t, x, z have no outgoing edges. This check includes every edge in the supplied graph. A DAG may have several valid topological orders; the displayed order is one of them.

Dijkstra1 (p. 7): Complete the shortest-path calculation

The source is a . The supplied edge $b-h$ has weight **1**; use this value rather than the weight in a different textbook version of this graph. Set $d(a) = 0$ and every other tentative distance to ∞ .

Use the relaxation rule $d(v) \leftarrow \min\{d(v), d(u) + w(u, v)\}$. At each step, settle the unsettled vertex with the smallest tentative distance. Break the c/i tie alphabetically.

Step	Settled	a	b	h	g	f	c	i	e	d
0	—	0	∞	∞	∞	∞	∞	∞	∞	∞
1	a	0	4	8	∞	∞	∞	∞	∞	∞
2	b	0	4	5	∞	∞	12	∞	∞	∞
3	h	0	4	5	6	∞	12	12	∞	∞
4	g	0	4	5	6	8	12	12	∞	∞
5	f	0	4	5	6	8	12	12	18	22
6	c	0	4	5	6	8	12	12	18	19
7	i	0	4	5	6	8	12	12	18	19
8	e	0	4	5	6	8	12	12	18	19
9	d	0	4	5	6	8	12	12	18	19

The important updates after the illustrated prefix are:

$$d(g) = d(h) + 1 = 6,$$

$$d(i) = d(h) + 7 = 12,$$

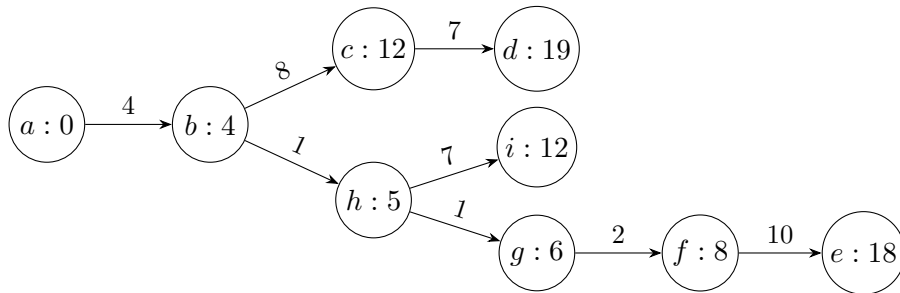
$$d(f) = d(g) + 2 = 8,$$

$$d(e) = d(f) + 10 = 18,$$

$$d(d) = d(f) + 14 = 22 \quad \text{initially,}$$

$$d(d) = \min(22, d(c) + 7) = \min(22, 19) = 19.$$

Settling i, e, d causes no further improvements. The final shortest-path tree is:



Each node label gives the vertex and its final distance. The source-to-vertex paths are read directly from this tree.

AStar1 (p. 8): A* with $h(B) = 16$

Let $g(n)$ be the best known path cost from A , and let $f(n) = g(n) + h(n)$. The heuristic values are

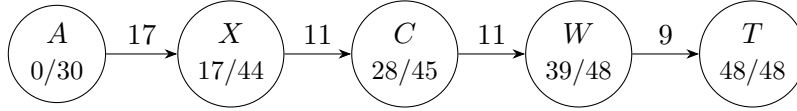
n	A	X	B	C	L	W	K	T	S	H
$h(n)$	30	27	16	17	33	9	19	0	17	22

Start with OPEN = $\{A\}$, $g(A) = 0$, and $f(A) = 30$. In the table, $n(g, f)$ denotes a queued node with its current g and f values. OPEN is shown in increasing f order after each expansion.

Step	Pop	Calculations	OPEN after step
1	A	$g(X) = 0 + 17 = 17$, $f(X) = 17 + 27 = 44$	$X(17, 44)$
2	X	$g(B) = 17 + 16 = 33$, $f(B) = 49$; $g(C) = 17 + 11 = 28$, $f(C) = 45$	$C(28, 45)$, $B(33, 49)$
3	C	$g(W) = 28 + 11 = 39$, $f(W) = 48$; $g(L) = 28 + 20 = 48$, $f(L) = 81$	$W(39, 48)$, $B(33, 49)$, $L(48, 81)$
4	W	$g(T) = 39 + 9 = 48$, $f(T) = 48$; $g(K) = 39 + 10 = 49$, $f(K) = 68$; candidate $g(B) = 39 + 12 = 51 > 33$, so keep B unchanged.	$T(48, 48)$, $B(33, 49)$, $K(49, 68)$, $L(48, 81)$
5	T	T is popped as the lowest- f node. It is the goal, so stop.	$B(33, 49)$, $K(49, 68)$, $L(48, 81)$

The CLOSED sequence is A, X, C, W, T . Edges back to previously processed nodes do not improve their costs in this run. Follow the parent pointers:

$$\boxed{A \rightarrow X \rightarrow C \rightarrow W \rightarrow T}, \quad \boxed{17 + 11 + 11 + 9 = 48}.$$



The pair under each node is g/f . The goal is tested when it is popped, not immediately when it is first generated.

AStar2 (p. 9): Change $h(B)$ from 16 to 6

Use the same initialization and relaxation rule, with $h(B) = 6$. Whenever a better route to a node already in OPEN is found, update its g , f , and parent, and restore the priority order.

Step	Pop	Calculations	OPEN after step
1	A	$g(X) = 17, f(X) = 44$	$X(17, 44)$
2	X	$g(B) = 33, f(B) = 33 + 6 = 39;$ $g(C) = 28, f(C) = 45$	$B(33, 39), C(28, 45)$
3	B	$g(W) = 33 + 12 = 45, f(W) = 54;$ $g(T) = 33 + 17 = 50, f(T) = 50;$ $g(S) = 33 + 7 = 40, f(S) = 57$	$C(28, 45), T(50, 50),$ $W(45, 54), S(40, 57)$
4	C	Candidate $g(W) = 28 + 11 = 39 < 45$: update $W, f(W) = 48, \text{parent}(W) = C$. Also $g(L) = 48, f(L) = 81$.	$W(39, 48), T(50, 50),$ $S(40, 57), L(48, 81)$
5	W	Candidate $g(T) = 39 + 9 = 48 < 50$: update $T, f(T) = 48, \text{parent}(T) = W$. Also $g(K) = 49, f(K) = 68$. Candidate $g(B) = 51 > 33$ is discarded.	$T(48, 48), S(40, 57),$ $K(49, 68), L(48, 81)$
6	T	Goal popped; stop.	$S(40, 57), K(49, 68),$ $L(48, 81)$

Thus the correct A* calculation still returns

$$A \rightarrow X \rightarrow C \rightarrow W \rightarrow T \text{ with cost } 48.$$

The expansion order changes to A, X, B, C, W, T , but the optimal path does not.

What can go wrong? If an implementation refuses to update a node already in OPEN, then after expanding C , W incorrectly remains at $g = 45, f = 54$. It next pops T with cost 50 and returns $A \rightarrow X \rightarrow B \rightarrow T$, which is suboptimal. Similarly, stopping when T is first generated is incorrect. Successors need cost relaxation, not merely insertion or a visited test.

Admissibility and consistency. Both 16 and 6 underestimate the true remaining cost from B , which is 17, so both are admissible. However, the new heuristic is inconsistent, since

$$h(X) = 27 > 16 + h(B) = 22, \quad h(S) = 17 > 7 + h(B) = 13.$$

For general graphs with an admissible but inconsistent heuristic, a CLOSED node must be reopened when a strictly better g is found. In *this particular run*, no CLOSED node receives a better cost: the necessary corrections concern W and T while they are still in OPEN.

AStar3 (p. 10): Shortest paths among polygonal obstacles

(a) **Number of states.** If a state is an arbitrary collision-free position (x, y) in the plane, there are *uncountably infinitely many* states. The coordinates vary continuously; the drawing is not a finite grid. There are also infinitely many possible continuous paths to the goal.

An equivalent finite shortest-path search can use the visibility graph: keep only S , G , and polygon vertices, connecting pairs that can see one another without crossing an obstacle interior. The seven pictured obstacles have

$$5 + 3 + 4 + 4 + 4 + 3 + 6 = 29 \text{ vertices.}$$

Including S and G gives 31 visibility-graph vertices. The continuous state space and this reduced search graph are different representations; 31 is not the number of all physical positions.

(b) **Why can the shortest path follow polygon boundaries?** Between two fixed points in free space, a straight segment is shortest. A bend strictly inside free space can be replaced locally by a straight chord, shortening the path. A non-collinear bend at the interior of a straight obstacle edge can also be straightened within the free half-plane. Therefore, essential direction changes occur at obstacle vertices.

A shortest path is consequently made of straight segments joining S , G , and some obstacle vertices. Some segments can coincide with polygon edges, while others cross free space between visible vertices. It is not necessary for every segment of the path to lie on an obstacle boundary.

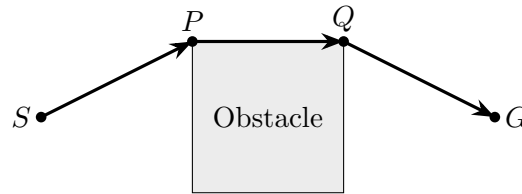


Illustration of the geometric principle, not the original scene

Assume a point robot, Euclidean path length, and that travel along obstacle boundaries is allowed. If contact is forbidden, boundary-hugging paths give the limiting infimum and can be approximated with arbitrarily small clearance.